

resolve (runner patch) – guide

Revision: 1 **Last modified:** 2026-09-26T14:05:00Z

Source: `constitution/scripts/codegraph/runner_patches/resolve1.py` (116 lines). A **module driven by** `fk_index_patch.py`, not a command. Citations are `resolve1.py:<line>` unless another file is named. Machinery guide: [fk_index_patch](#).

Overview

Stock CodeGraph 1.6.0 `ReferenceResolver.createContext().getAllFiles` (`lib/dist/resolution/index.js`) runs `SELECT path FROM files ORDER BY path` and maps the rows on **every call**. Several per-reference paths call it, most importantly the vue framework's `resolveComponent`, reached for every `calls` reference with a PascalCase name in any language once vue is detected (one `.vue` file anywhere suffices). On a 584,603-file index the source records ~350 ms per call and 93.4% of resolve wall time (real-repository profile), i.e. $O(\#refs \times \#files)$ (docstring lines 3-11).

Fix (docstring lines 13-27; five anchors in two files):

- `getAllFiles` returns `this.helixAllFilesMemo`, filled on first use with the unchanged query result (same `ORDER BY path` order) and `Object.freeze()`d, so callers see one stable array and any in-place mutation throws instead of corrupting the memo (source states no stock caller mutates it, audited 2026-09-24).
- The memo is dropped in `clearCaches()`, the same invalidation point stock uses for `knownFiles` and other caches.
- The memo is also dropped at the top of `initialize()`, because stock `initialize()` runs `detectFrameworks()` before its `clearCaches()`, and `detect()` reads `getAllFiles()`.

- In `frameworks/vue.js` a `WeakMap` keyed on the (identity-stable) file list caches the `.vue` subset, so `resolveComponent` no longer walks every file per reference; `filter` preserves order, so matches and tie-breaks are element-for-element what stock computed (lines 62-99).

Honest boundary (source lines 29-33): between two `clearCaches()` calls the list is a snapshot; stock re-queried. Every stock path that changes the `files` table reaches `clearCaches()` / `initialize()` before resolving, and the resolver never writes the `files` table.

Prerequisites

Reads `lib/dist/resolution/index.js` and `lib/dist/resolution/frameworks/vue.js`. Python stdlib plus `runner_patches/common.py`. Default-set patch (`runner_patches/__init__.py:17`), applied fourth. It is one of the two patches `codegraph_safe.sh` uses by default (`--patches fkidx1,resolve1` , per the parent guide).

Usage

Only through `fk_index_patch.py` :

```
python3 scripts/codegraph/fk_index_patch.py --patches resolve1 --dst <scratch dir>
```

- VERIFIED (ran: `tests/test_resolve1.sh` (drives `fk_index_patch.py`) with `TMPDIR` set to a scratch directory; saw: `SUMMARY cases=7 failed=0, exit 0`) . The test builds a `--patches resolve1` runner under its own workdir from the installed stock package (~282 MB copy) and never touches the real cache (`HELIX_CODEGRAPH_RUNNER_DIR` is set to its workdir, `test_resolve1.sh:45`).
- VERIFIED (ran: `resolve1.plan` against stock 1.6.0 via `fk_index_patch.plan_all`; saw: `APPLIED, files lib/dist/resolution/index.js` and `lib/dist/resolution/frameworks/vue.js`, five anchors, detail `memo_field helixAllFilesMemo, frozen true`) .

Behaviour (source-traced)

`plan(view)` (lines 102-116):

1. Refuse `resolve1`: source is ALREADY patched – --src must be the pristine package if `helixAllFilesMemo` occurs in `resolution/index.js` or `helixVueSubsetCache` in `vue.js` (lines 103-105).
2. Five `replace_once` edits (lines 107-111): `getAllFiles` (memoized and frozen), `clearCaches` reset, `initialize()` entry reset, the vue helper `helixVueFiles`, and the vue loop for `(const file of helixVueFiles(context.getAllFiles()))` (which drops the per-file `endsWith('.vue')` test because the subset is already filtered).
3. Any anchor that does not occur exactly once raises Refuse (`common.py`), exit 2, no runner. There is no NOT_NEEDED path in this patch (no such branch in the source).

3 2 0

What the test proves (`tests/test_resolve1.sh` , header lines –)

R1 `getAllFiles()` returns one frozen array per generation equal to the query result; R2 `clearCaches()` invalidates it; R3 `initialize()` invalidates it before `detectFrameworks()`; R4 real `getAllFilePaths()` queries during a full index do not grow with the number of references (bounded by `R4_MAX`, default 12); R5 the `.vue` subset is derived once per file-list identity; R6 and R7 resolution output (sorted edge dump sha256) is byte-identical to the stock package on a mixed-language fixture with vue and on a vue-free fixture. Companion files:

`test_resolve1_probe.js`, `_drive.js`, `_count_hook.js`, `_dump.py`, `_fixture.py`, `_equiv_real.js`; `test_resolve1_mutations.sh` applies single-edit mutants of the patch (table in `test_resolve1_mutate.py`) and requires the named tests to turn red.

Edge cases

- Already-patched input: VERIFIED (ran: `plan` on the patched text; saw: Refuse "source is ALREADY patched") (parent guide, Edge cases).

- Any anchor absent or duplicated: VERIFIED (parent guide: for every anchor of `resolve1`, removing or duplicating it in a copy of the real 1.6.0 files produced `Refuse "anchor '<label>' occurs 0x|2x ... expected exactly 1")` .
- A code path that mutates the returned array now throws (`Object.freeze`); the docstring says no stock caller does.
- Files added to the `files` table between two `clearCaches()` calls are not seen by `getAllFiles` ; this is the snapshot boundary above.

Related scripts

[fk_index_patch](#), siblings [fkidx1](#), [lockfix1](#), [datafrag1](#), [resolve2](#) (reads the same `resolution/index.js`), [mcpro1](#), [codegraph_safe](#).

Last verified

2026-09-26: `tests/test_resolve1.sh` 7/7 PASS with a scratch `TMPDIR` ; the real runner cache directory listing was unchanged (`1.6.0-fkidx1` , `1.6.0-fkidx1-resolve1` , `1.6.0-mcpro1`). `tests/test_resolve1_mutations.sh` : SUMMARY cases=6 failed=0 (golden plus 5 mutants, each required red and killed).